

Laboratorio N°1

Joaquín Benítez

Ingeniería Mecatrónica

Universidad Tecnológica del Uruguay

Mercedes, Uruguay

joaquin.benitez@estudiantes.utec.edu.uy

Agustín Montero

Ingeniería en Mecatrónica

Universidad Tecnológica del Uruguay

Mercedes, Uruguay

agustin.montero@estudiantes.utec.edu.uy

Santiago Soria

Ingeniería en Mecatrónica

Universidad Tecnológica del Uruguay

Dolores, Uruguay

santiago.soria@estudiantes.utec.edu.uy

Abstract—En este informe se presenta el diseño, simulación e implementación física de cuatro sistemas digitales sobre el microcontrolador ATmega328P. El trabajo abarca el desarrollo de máquinas de estados (FSM), una unidad lógica aritmética (ALU) y el uso integral del lenguaje ensamblador AVR, validando el correcto funcionamiento de cada módulo tanto en simulación como en hardware.

Index Terms—ATmega328P, ALU, FSM, Lenguaje Ensamblador, GPIO, SREG

I. INTRODUCCIÓN

Este documento detalla el desarrollo del primer laboratorio, el cual se enfoca en la programación a bajo nivel sobre la arquitectura AVR del microcontrolador ATmega328P, abarcando el estudio y manipulación directa de sus recursos internos. Partiendo del uso del lenguaje ensamblador (ASM), el trabajo profundiza en la configuración de puertos de entrada/salida general (GPIO), la gestión de registros de propósito general, el monitoreo de banderas en el registro de estado (SREG), la implementación de estructuras de salto y la generación de *delays* temporales por software. Como parte del desarrollo experimental, se diseñan e implementan diversos sistemas digitales, incluyendo un contador con tres botones y visualización en display de 7 segmentos con filtrado de antirrebote por software, un secuenciador de patrones luminosos sobre 8 LEDs controlado mediante máscaras de bits y rotaciones, y una Unidad Aritmético-Lógica (ALU) de 4 bits con actualización de banderas de estado (C, N y Z). Adicionalmente, se diseña e implementa la lógica de control para una lavadora automática mediante el modelado de Máquinas de Estados Finitos (FSM), integrando la lectura de sensores, la parametrización de tiempos según la carga y la notificación de estados mediante LEDs. Finalmente, la validez de los algoritmos y diseños propuestos es verificada exitosamente tanto a través de simulaciones previas como mediante el montaje práctico en hardware y el versionado de código en GitHub.

II. OBJETIVOS

A. Objetivo general

Diseñar, simular e implementar sistemas digitales de bajo nivel utilizando lenguaje ensamblador AVR en el microcontrolador ATmega328P, integrando el control directo de periféricos de entrada/salida (GPIO), máquinas de estados finitos (FSM), la lógica de una Unidad Aritmético-Lógica (ALU) y la gestión de banderas en el registro de estado SREG.

B. Objetivos específicos

- Programar los puertos de entrada/salida (GPIO) en ensamblador AVR para controlar el conteo en display de 7 segmentos con filtrado antirrebote y un secuenciador de 8 LEDs mediante máscaras de bits.
- Implementar una Máquina de Estados Finitos (FSM) para el control de una lavadora automática y una Unidad Aritmético-Lógica (ALU) de 4 bits con gestión de banderas en el registro SREG.
- Validar el correcto funcionamiento de los sistemas digitales mediante simulación previa, montaje en hardware e integración de control de versiones en GitHub.

III. MATERIALES UTILIZADOS

A. Componentes (Hardware)

- Arduino UNO
- Microcontrolador ATmega328P
- Protoboard
- Fuente de alimentación
- LEDs
- Display BCD de 7 segmentos
- Resistencias
- Pulsadores
- Cables

B. Programas (Software)

- Microchip Studio
- PcsinLab
- Git y GitHub

IV. MARCO TEÓRICO

A. Arquitectura AVR y Microcontrolador ATmega328P

El ATmega328P es un microcontrolador de 8 bits basado en la arquitectura RISC de AVR con estructura Harvard, la cual separa físicamente la memoria del programa (Flash de 32 KB) de la memoria de datos (SRAM de 2 KB) [6]. Posee 32 registros de propósito general directamente conectados a la Unidad Aritmético Lógica (ALU), permitiendo ejecutar la mayoría de las instrucciones en un único ciclo de reloj [6].

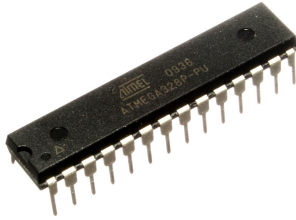


Fig. 1: Microcontrolador ATmega328P

B. Entradas Digitales, Resistencias Pull-Up y Lógica Active-LOW

Los microcontroladores AVR de 8 bits organizan sus pines digitales de entrada/salida general (GPIO) en grupos de hasta 8 bits denominados puertos (identificados por letras como Puerto B, C o D). Cada puerto x está controlado de manera independiente mediante un conjunto de tres registros de 8 bits situados en la memoria de E/S [7]:

1) *DDRx (Data Direction Register)*: Controla el sentido de dirección de cada pin n del puerto x ($DDxn$) [7].

- Un bit en '1' configura el pin como salida digital.
- Un bit en '0' configura el pin como entrada digital (estado por defecto tras un reinicio del sistema).

2) *PORTx (Pin Output Register)*: Posee una doble función dependiendo de la configuración del registro de dirección *DDRx* [7]:

- Como Salida ($DDxn = 1$): Escribir un '1' establece el pin físico en nivel alto (V_{CC}), mientras que escribir un '0' fuerza un nivel bajo (GND).
- Como Entrada ($DDxn = 0$): Escribir un '1' habilita de forma individual la resistencia interna de pull-up del pin. Si se escribe un '0', el pin queda en estado de alta impedancia (tri-state o flotante).

3) *PINx (Input Pins Address)*: Es un registro de solo lectura que refleja directamente el nivel lógico físico presente en las terminales externas del puerto x . En arquitectura AVR, escribir un '1' en un bit del registro $PINxn$ conmuta (toggle) automáticamente el valor correspondiente del registro $PORTxn$, independientemente de la dirección configurada [7].

4) *Configuración de Entradas con Pull-Up y Lógica Active-LOW*: Cuando un pin digital se configura como entrada sin activar su resistencia de pull-up (estado tri-state), la terminal física se vuelve vulnerable al ruido electromagnético ambiental y a las cargas estáticas circundantes, provocando estados flotantes e impredecibles en la lectura del registro $PINx$ [7].

Para resolver este inconveniente sin añadir componentes externos, la arquitectura AVR permite activar la resistencia

de pull-up integrada. Bajo esta configuración ($DDxn = 0$, $PORTxn = 1$), la resistencia conecta internamente la terminal a V_{CC} , fijando un nivel lógico ALTO ('1') en reposo. Al conectar pulsadores o sensores mecánicos (como el interruptor de puerta o nivel de agua) entre el pin y masa (GND), la pulsación o cierre del contacto deriva la corriente a tierra, forzando la entrada a un nivel lógico BAJO ('0') [7].

C. Registro de Estado SREG y Banderas Aritméticas (Flags)

El registro de estado SREG (*Status Register*) de 8 bits refleja los resultados de las operaciones aritméticas y lógicas ejecutadas por la ALU [9]. Entre sus banderas principales destacan [9]:

- **Acarreo (C - Carry Flag)**: Se activa si una operación aritmética genera un acarreo en el bit más significativo (MSB) [9].
- **Cero (Z - Zero Flag)**: Indica si el resultado de la última operación fue exactamente cero [9].
- **Negativo (N - Negative Flag)**: Copia el bit de mayor peso (MSB) del resultado, indicando un valor negativo en representación de complemento a dos [9].

D. Máquinas de Estados Finitos (FSM)

Una Máquina de Estados Finitos (FSM) es un modelo comportamental compuesto por un número finito de estados, transiciones y acciones [3]. En el modelo Moore, las salidas dependen exclusivamente del estado actual del sistema, mientras que en el modelo Mealy dependen tanto del estado como de las entradas [3]. Las FSM se implementan en lenguaje ensamblador mediante registros de estado y saltos condicionales (CPI, BREQ, RJMP), resultando indispensables para secuenciar etapas temporizadas como ciclos de lavado [3].

E. Interfaces de Salida Digital y Decodificación

La visualización mediante displays de 7 segmentos requiere la conversión de valores numéricos a patrones binarios específicos para cada segmento (de 'a' a 'g') [4]. En configuraciones de cátodo común, los segmentos se encienden con un nivel lógico alto (1), requiriendo una tabla de búsqueda (*Look-Up Table*) para decodificar los dígitos [4]. En paralelo, la secuencia de 8 LEDs se controla mediante operaciones de desplazamiento (LSL, LSR) y máscaras de bits (ANDI, ORI) [10].

F. Filtrado Antirrebote (Software Switch Debouncing)

Al pulsar un interruptor mecánico, las láminas metálicas rebotan momentáneamente antes de fijar el contacto, generando ruido eléctrico y falsas lecturas [1]. En microcontroladores, el filtrado antirrebote por software introduce un retardo o temporización de estabilización (10 ms a 50 ms) tras detectar la primera transición, verificando posteriormente la permanencia de la señal antes de confirmar la entrada. Para mitigar la inestabilidad de los contactos mecánicos se emplean soluciones tanto por hardware como por software. En el dominio del hardware, el problema suele resolverse añadiendo un filtro pasa-bajos

RC junto a un disparador Schmitt Trigger o mediante un arreglo *latch* RS , encargados de atenuar los picos de tensión de alta frecuencia antes de que la señal ingrese al puerto del microcontrolador. [1]

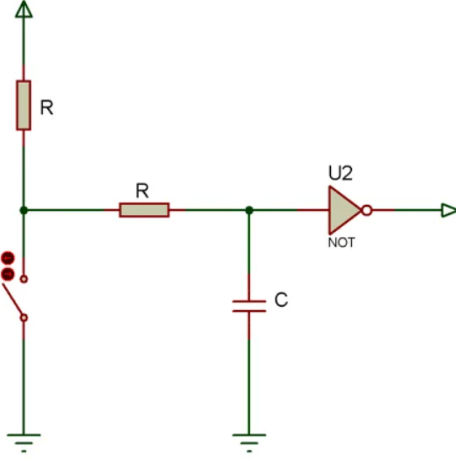


Fig. 2: Circuito RC Antirrebote

G. Instrucciones de Salto y Control de Flujo

El control de flujo en la arquitectura AVR se realiza mediante instrucciones de salto incondicional (RJMP, JMP) y comparaciones seguidas de saltos condicionales (CPI, BREQ, BRNE, BRCS) [5]. Estas instrucciones evalúan el estado de las banderas en el registro SREG tras una operación previa. Son fundamentales para la navegación entre menús mediante botones, la selección de modos de operación y la ejecución de las transiciones en las máquinas de estados finitos [5].

```
.include "m328pdef.inc"
.org 0x00

; Inicialización
ldi r16, 0x20
out DDRB, r16

main_loop:
sbi PORTB, 5
rcall delay

cbi PORTB, 5
rcall delay

rjmp main_loop

; Subrutina de retardo de 100 ms a 16 MHz
delay:
ldi r18, 21
ldi r19, 199
L1: dec r19
brne L1
dec r18
brne L1
ret
```

Fig. 3: Ejemplo de código en Lenguaje Ensamblador

H. Operaciones Aritmético-Lógicas en Ensamblador

La Unidad Aritmético-Lógica (ALU) del ATmega328P ejecuta operaciones matemáticas y lógicas bit a bit directamente sobre los registros de propósito general [2]. Entre las instrucciones fundamentales utilizadas se encuentran [5]:

- **Aritméticas:** ADD (suma), SUB (resta) e INC (incremento en 1) [5].
- **Lógicas:** AND (conjunción), OR (disyunción) y EOR (OR exclusiva / XOR) [5].
- **Manipulación directa:** CLR (limpieza/puesto a cero) y LSL (desplazamiento lógico a la izquierda) [5].

TABLE I: Tabla de funcionamiento y señales de selección de la ALU.

S_2	S_1	S_0	Operación / Resultado (F)
0	0	0	CLEAR (0000_2)
0	0	1	Resta ($A - B$)
0	1	0	Suma ($A + B$)
0	1	1	Lógica XOR ($A \oplus B$)
1	0	0	Lógica AND ($A \text{ AND } B$)
1	0	1	Lógica OR ($A \text{ OR } B$)
1	1	0	Desplazamiento ($A \ll 1$)
1	1	1	Incremento ($A + 1$)

I. Generación de Retardos por Software (Software Delays)

Debido a que las instrucciones en lenguaje ensamblador consumen un número conocido de ciclos de reloj (típicamente entre 1 y 3 ciclos), es posible generar pausas temporales precisas mediante ciclos o bucles anidados de conteo decreciente [8]. Cargar valores en registros de propósito general y decrementarlos repetidamente (DEC, BRNE) permite pausar la ejecución del programa el tiempo requerido para el control de tiempos de lavado, giros de tambor o temporización entre secuencias de LEDs [8].

V. METODOLOGÍA

A. Problema A: Control de lavadora automática

Para la primera parte del laboratorio, centrada en el diseño de un sistema de control para una lavadora automática mediante el microcontrolador ATmega328P, se procedió al desarrollo del firmware exclusivamente en lenguaje ensamblador (ASM). Inicialmente, se configuraron los puertos de entrada para la lectura del pulsador de inicio, el selector de carga, el sensor de seguridad de la puerta y el sensor de llenado de agua. Posteriormente se establecieron los puertos de salida destinados al accionamiento del motor y a la activación de ocho LEDs indicadores del estado del proceso (Standby, Lavado, Centrifugado, Secado, Fin de proceso) y del nivel de carga (Ligera, media, pesada). La lógica de control se modeló mediante una máquina de estados finitos (FSM) que gestiona la duración y dirección de los giros del tambor para las etapas de lavado, centrifugado y secado. Para adaptar el ciclo al tipo de carga detectada, se implementaron rutinas paramétricas de retardo por software que incrementan los tiempos base de giro y pausa preestablecidos para cada etapa. Finalmente, el código fue simulado para validar las transiciones lógicas y temporales antes de su implementación física.

B. Problema B: Contador digital con display de 7 segmentos

Para la segunda problemática, orientada a la visualización numérica y el control de eventos, se implementó un contador digital comandado por tres pulsadores de entrada destinados a incrementar, decrementar y reiniciar el conteo. El desarrollo en lenguaje ensamblador incluyó la configuración de los puertos del microcontrolador y el diseño de rutinas de lectura de botones que integran técnicas de eliminación del efecto de rebote mediante software para garantizar transiciones limpias y libres de falsos disparos. Simultáneamente, se construyó la lógica de decodificación necesaria para transformar el valor del registro numérico interno a los patrones de bits correspondientes al display de 7 segmentos. La arquitectura del programa se estructuró para mantener un refresco continuo en el puerto de salida, asegurando que el valor actualizado del contador se visualice permanentemente a lo largo de toda la ejecución y verificando su correcto comportamiento mediante simulación antes del montaje experimental.

C. Problema C: Selección y ejecución de secuencias luminosas

En el desarrollo del tercer problema, focalizado en el manejo avanzado de salidas digitales, se programó un sistema selector de ocho secuencias de luz empleando un puerto completo del microcontrolador conectado a un arreglo de 8 LEDs. Se configuraron tres pines de entrada para la navegación entre los distintos patrones visuales, permitiendo avanzar hacia la siguiente secuencia, retroceder a la anterior o retornar directamente a la primera de ellas. La lógica interna se basó en una arquitectura de código modular sustentada en máscaras de bits, operaciones de desplazamiento de registros y saltos condicionales para reproducir de forma cíclica los patrones luminosos. Para garantizar una respuesta inmediata a los comandos del usuario, el escaneo de los pulsadores se intercaló estratégicamente dentro de los bucles de retardo por software, lo que permitió abortar la secuencia activa y ejecutar el cambio de rutina de forma fluida sin requerir displays adicionales.

D. Problema D: Unidad Aritmético-Lógica (ALU) de 4 bits

Para la cuarta problemática, correspondiente al diseño de una Unidad Aritmético-Lógica (ALU), se programó el microcontrolador en lenguaje ensamblador para recibir a través de sus puertos de entrada dos operandos de 4 bits (A y B) junto con un selector de 3 bits (S). El núcleo del sistema consistió en rutear el flujo de ejecución mediante la evaluación de la señal de selección para realizar la operación matemática o lógica solicitada: borrado (CLEAR), resta, suma, XOR, AND, OR, desplazamiento lógico a la izquierda (SHL) o incremento. Tras la ejecución, el programa fue diseñado para extraer la información pertinente del registro de estado (SREG) y actualizar físicamente las banderas de Acarreo (C), Cero (Z) y Negativo (N). Finalmente, se asignó el resultado numérico de la operación a los puertos de salida correspondientes, acompañado de indicadores visuales del código de selección en curso, culminando con la validación virtual y física de todo el sistema integrado.

VI. RESULTADOS

A. Parte A

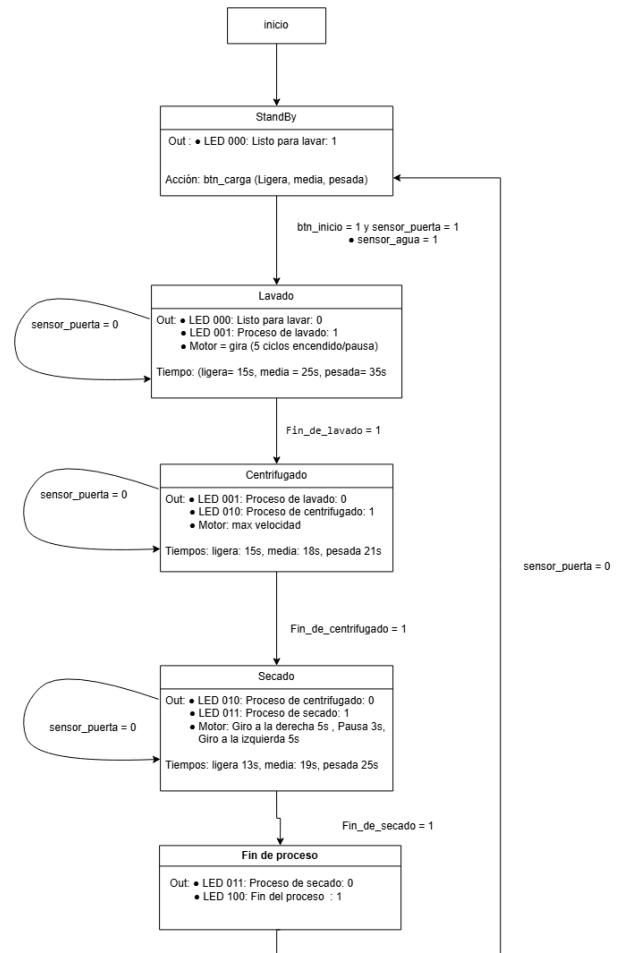


Fig. 4: Circuito de la parte A

1) *Maquina FSM:* El sistema implementa una máquina de estados finita secuencial de 5 estados fundamentales (StandBy, Lavado, Centrifugado, Secado y Fin de proceso) para el control automatizado del ciclo del equipo. El flujo inicia en el estado de reposo (StandBy), donde el indicador LED 000 confirma que el equipo está listo y se parametriza la carga (Ligera, Media o Pesada). La transición hacia el estado de Lavado se ejecuta condicionalmente al verificar la activación del botón de arranque y las señales lógicas válidas de los sensores de seguridad y nivel ($btn_inicio = 1$, $sensor_puerta = 1$ y $sensor_agua = 1$). Durante las etapas operativas de Lavado (con motor en 5 ciclos encendido/pausa), Centrifugado (motor a máxima velocidad) y Secado (patrón de giro alternado a la derecha, pausa y giro a la izquierda), los tiempos de ejecución se adaptan según el tipo de carga seleccionada y se activan secuencialmente los indicadores LED 001, LED 010 y LED 011 respectivamente. Además, cada etapa activa cuenta con un bucle de seguridad/pausa activado al abrir la puerta ($sensor_puerta = 0$). Las transiciones entre etapas se realizan al cumplirse las banderas temporizadas de

finalización (Fin_de_lavado, Fin_de_centrifugado y Fin_de_secado), culminando en el estado Fin de proceso (LED 100), desde el cual el sistema reajusta sus salidas y retorna automáticamente a StandBy al detectar la apertura de la puerta.

2) Simulación: .

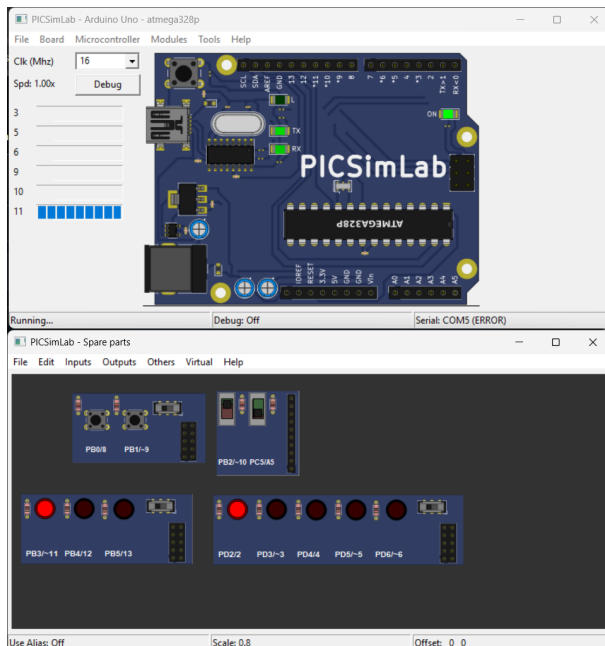


Fig. 5: Simulación parte A

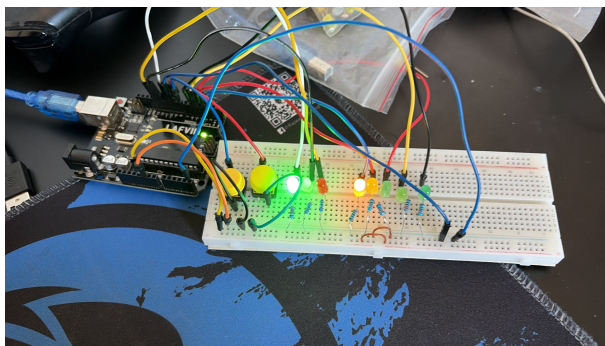


Fig. 6: Circuito de la parte A

B. Parte B

Se logró el control preciso de un display de 7 segmentos para el contador digital gestionado por tres pulsadores mediante programación exclusiva en lenguaje ensamblador (ASM). Se desarrollaron rutinas efectivas para eliminar el efecto rebote de los botones, permitiendo que el primer botón incremente el valor, el segundo lo decremente y el tercero reinicie el contador a su valor inicial. El sistema demuestra una correcta configuración de los puertos de entrada/salida y la exitosa conversión de valores numéricos a los patrones del display.

1) simulación: .

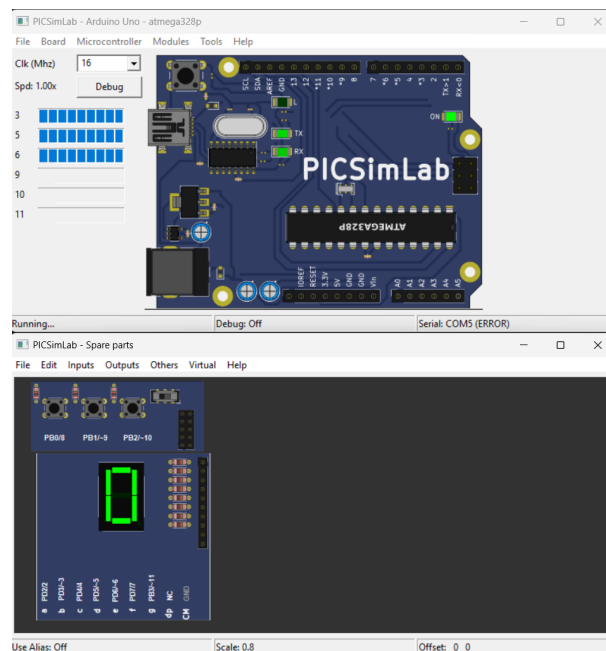


Fig. 7: Simulación parte A

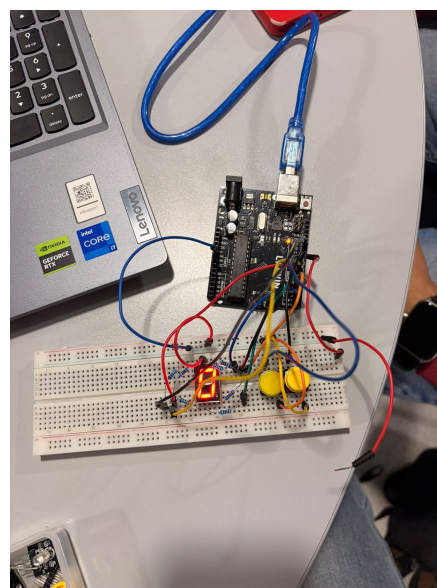


Fig. 8: Circuito de la parte B

C. Parte C

El selector de secuencias luminosas opera de manera continua sobre 8 LEDs reflejando distintos patrones de encendido y apagado. La implementación se estructuró de forma modular utilizando operaciones de desplazamiento, máscaras de bits y saltos condicionales para alternar entre ocho secuencias distintas sin emplear un display para indicar el estado. Mediante la lectura de tres botones, el sistema permite avanzar a la siguiente secuencia, retroceder a la anterior o regresar directamente a la secuencia inicial.

1) *simulación:* .

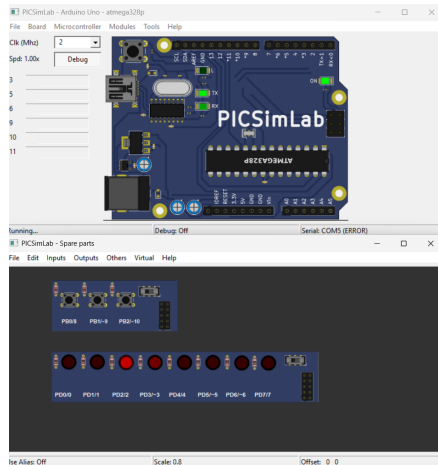


Fig. 9: Simulación parte C

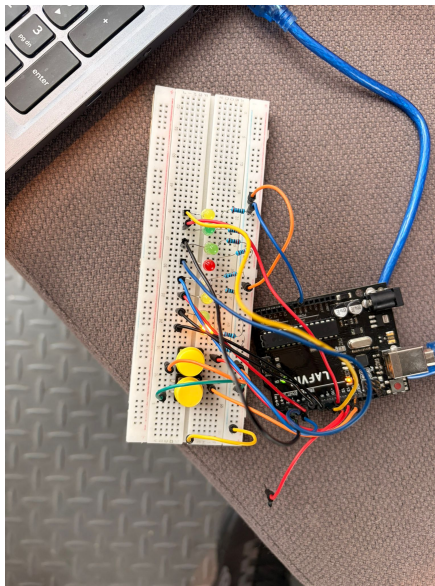


Fig. 10: Circuito de la parte C

D. Parte D

La Unidad Lógica Aritmética (ALU) se programó en el microcontrolador ATmega328P para procesar dos operandos de 4 bits (A y B) basándose en una señal selectora de 3 bits. El sistema logra ejecutar exitosamente las ocho operaciones implementadas: CLEAR, $A - B$, $A + B$, XOR, AND, OR, desplazamiento a la izquierda ($A \ll 1$) e incremento ($A + 1$). Como resultado adicional, la ALU actualiza correctamente las banderas de estado Carry (C), Negativo (N) y Zero (Z) tras cada cálculo, e incluye una salida que identifica visualmente la operación que se está ejecutando.

1) *simulación:* .

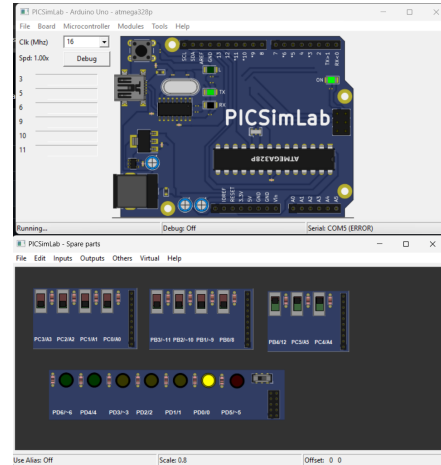


Fig. 11: Simulación parte C

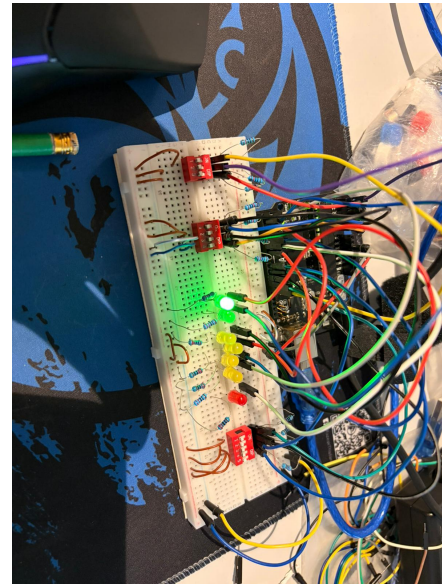


Fig. 12: Circuito de la parte C

VII. CONCLUSIONES

El presente trabajo de laboratorio ha permitido consolidar los conocimientos teóricos y prácticos relacionados con la arquitectura del microcontrolador ATmega328P, destacando la importancia y el rigor de la programación a bajo nivel mediante el lenguaje ensamblador. La interacción directa con los recursos físicos del dispositivo facilitó una comprensión profunda de la organización de sus registros de propósito general, la configuración precisa de los puertos de entrada/salida (GPIO) y la estructuración del código para manipular el hardware con alta eficiencia. Mediante la resolución de los distintos escenarios planteados, se demostró la viabilidad de diseñar e implementar sistemas de control basados en máquinas de estados finitos. La automatización del proceso de una lavadora y el desarrollo del selector de secuencias luminosas evidenciaron

la necesidad de estructurar los programas de forma modular. Para lograr esto, fue indispensable el dominio de instrucciones de salto condicional e incondicional, la creación de rutinas de retardo por software, las operaciones de desplazamiento y el uso riguroso de máscaras de bits. Asimismo, el manejo y acondicionamiento de señales físicas quedó patente en el desarrollo del contador digital, donde la eliminación del efecto rebote de los pulsadores y la conversión algorítmica de valores para su correcta visualización en un display de 7 segmentos resultaron fundamentales para garantizar la estabilidad de la interfaz. Por otro lado, el diseño y la construcción de una Unidad Lógica Aritmética (ALU) constituyeron un ejercicio crucial para el entendimiento del procesamiento interno de datos a nivel de máquina. La implementación de esta unidad requirió la ejecución controlada de operaciones aritméticas y booleanas sobre operandos de 4 bits, estrictamente guiadas por señales de selección de 3 bits. Este apartado afianzó de manera definitiva el manejo del registro de estado (SREG), demostrando cómo las banderas de acarreo (Carry), resultado nulo (Zero) y valor negativo (Negativo) interactúan dinámicamente tras cada instrucción, proporcionando la retroalimentación técnica necesaria sobre los cálculos ejecutados por el procesador.

VIII. APÉNDICE

A. *Link a Google Drive*

Haz clic aquí para ir a la carpeta en Google Drive.

B. *Link a repositorio de Github*

Haz clic aquí para ir al repositorio de Github.

REFERENCES

- [1] IIES Bangalore. Switch debouncing in microcontrollers, 2024.
- [2] All About Circuits. Building an arithmetic logic unit (alu), 2023.
- [3] All About Circuits. Finite state machines in sequential circuits, 2024.
- [4] GeeksforGeeks. Seven segment displays configuration and working, 2025.
- [5] Microchip Technology. Avr instruction set manual, 2022.
- [6] Microchip Technology. Atmega328p 8-bit avr microcontroller datasheet, 2023.
- [7] Microchip Technology Inc. AVR® MCU Digital I/O Ports. <https://developerhelp.microchip.com/xwiki/bin/view/products/mcu-mpu/8-bit-avr/structure/ioports/>, November 2023. Developer Help. Última modificación: 9 de noviembre de 2023. Accedido el 29 de agosto de 2026.
- [8] RJHcoding. Avr tutorials - delay loops in assembly, 2021.
- [9] RJHcoding. Avr tutorials - the status register (sreg), 2021.
- [10] ShareTechnote. Gpio (general purpose input output), 2024. Accessed: 2024-06-01.